

Bases à avoir :

Bases sur les dockers

Base de ros2 et de son fonctionnement

Savoir à peu près comment fonctionne le robot (VS Code, Hôte /docker)

Connaitre l'existence et savoir un peu utiliser Teltonika et noVNC

TP :

Expliquer ce qu'on va faire

Ouvrir vs code dans la racine du roskit_ws

Installation des bibliothèques

Ajout du GPU

Code .py pour vérifier le GPU

Création du package de détection

Création du code

Ajout du setup.py

On complète de package.xml

On fait la manipulation pour lancer -> ça marche

Intro :

Dans ce TP le but va être de faire de la détection d'objets grâce à YoloV11n. Le robot est équipé du CPU Intel Ultra 7 255H. Il possède une unité graphique intégrée. Notre but va être de l'utiliser. Grâce à elle on va pouvoir décupler les performances de Yolo. Or le docker n'a de base pas accès à cette unité graphique. Dans un environnement Docker, par sécurité, le container est isolé du matériel de l'hôte. Pour qu'il puisse "toucher" au GPU Intel, il faut lui ouvrir une porte. La première partie du TP va donc être de permettre au docker d'y avoir accès. Pour pouvoir mieux gérer les calculs on va utiliser openVINO qui est un toolkit développé par intel spécialement pour les calculs CPU :

OpenVINO (Open Visual Inference and Neural network Optimization) est un toolkit gratuit développé par **Intel**. Son but est simple : faire en sorte que tes modèles d'IA (comme YOLO) tournent le plus vite possible sur du matériel Intel (ton processeur Ultra 7, ton GPU intégré Arc, et ton NPU).

C'est un peu comme un "super-traducteur" et un "optimiseur" de code pour l'IA.

1. Comment ça fonctionne ?

Quand tu entraînes un modèle avec PyTorch (le format `.pt` de YOLO), le code est écrit de manière très générale. OpenVINO prend ce modèle et le transforme en deux étapes :

- **Le Model Optimizer** : Il simplifie l'architecture du réseau de neurones. Par exemple, il fusionne certaines couches mathématiques pour réduire le nombre de calculs sans perdre en précision.
- **L'Inference Engine** : C'est le moteur qui va lire ce modèle optimisé. Il est capable de dire : *"Tiens, cette partie du calcul ira plus vite sur le GPU Arc, et cette autre partie sera gérée par le NPU"*.

2. Pourquoi c'est crucial pour ton robot ?

Sans OpenVINO, ton robot utiliserait uniquement la puissance brute de ton CPU, ce qui est "lourd" et fait chauffer la machine. Avec OpenVINO :

- **Gain de FPS** : Tu peux passer de 10 FPS à 40 ou 50 FPS sur ton Ultra 7.
- **Économie d'énergie** : En utilisant le GPU et le NPU, ton CPU reste disponible pour gérer la navigation (Nav2), le Lidar, et les capteurs du robot.
- **Latence réduite** : Le temps entre le moment où la caméra voit un panneau "Stop" et le moment où l'IA le reconnaît est beaucoup plus court.

Dans un deuxième temps il faudra mettre en place l'environnement de travail ROS2 (création des packages, ajout des codes, importation du modèle, etc.)

Le rendu final se fera sur noVNC ou on aura un retour vidéo de la caméra et de la détection d'objets.

Partie 1 :

- Connecter le robot en ssh à vscode
- Ouvrir vscode à la racine du projet donc à `roskit_ws`
- On va configurer l'environnement pour avoir accès au GPU intégré du CPU Intel :
 - Dans le docker, Linux gère les permissions avec un système d'utilisateurs et de groupes. Chaque fichier, device ou ressource appartient à un utilisateur et un groupe. Pour accéder à une ressource, soit tu es le propriétaire, soit tu es dans le bon groupe.
 - Le groupe `render` est le groupe du GPU intel c'est donc celui la qu'il faut utiliser. Or le docker n'a pas acces à ce groupe, il n'a pas les permissions et n'est pas defini dans les fichiers du docker. Il faut donc l'ajouter manuellement.

- Avec la commande : `cat /etc/group` on peut voir tous les groupes du système. Ainsi on peut trouver l'id du groupe render du CPU :

```
render:x:992:user
```

Avec la commande : `stat /dev/dri/renderD128` on peut voir plus en détail ce groupe :

```
user@grkit-009:~/roskit_ws$ stat /dev/dri/renderD128
File: /dev/dri/renderD128
Size: 0          Blocks: 0          IO Block: 4096   character special file
Device: 0,6      Inode: 920          Links: 1        Device type: 226,128
Access: (0660/crw-rw----)  Uid: (  0/   root)   Gid: ( 992/  render)
Access: 2026-04-28 13:59:39.202910261 +0200
Modify: 2026-04-28 13:59:39.202910261 +0200
Change: 2026-04-28 13:59:40.661999607 +0200
Birth: 2026-04-28 13:59:38.996999734 +0200
```

En faisant cette commande on voit que le render est sur le groupe 992 : lors de l'installation des drivers sur l'hôte Ubuntu 24.04 (noble) Linux a créé ce groupe automatiquement et lui a assigné le GID (Group ID) 992. Ce numéro est attribué dynamiquement — sur une autre machine il pourrait être différent.

Dans le conteneur Ubuntu 22.04 (jammy), le groupe render existe aussi mais il a été créé par Ubuntu avec le GID 109 par défaut — parce que jammy et noble assignent les GIDs différemment, de manière totalement indépendante.

Donc il y a un problème de Gid qu'il faut corriger pour donner l'accès.

- Première étape :
Ajouter ces deux lignes dans le **Dockerfile** (dans `/roskit_ws/src/gr_docker`, c'est là que la configuration du docker se fait) avant des dependencies :

```
ARG RENDER_GID=#your GID
RUN groupmod -g ${RENDER_GID} render || groupadd -g ${RENDER_GID} render
```

Elles permettent de synchroniser le GID avec l'hôte

```
# Synchronise le GID render avec l'hôte
ARG RENDER_GID=992
RUN groupmod -g ${RENDER_GID} render || groupadd -g ${RENDER_GID} render

##### ADD NEW DEPENDENCIES JUST ABOVE THIS LINE #####
```

- Deuxième étape :
Ajouter la ligne, toujours dans le **Dockerfile** :

```
RUN usermod -a -G render $USER
```

Avec cette ligne on ajoute l'utilisateur au groupe render

```

RUN useradd -ms /bin/bash -u $UID -g $GID $USER
RUN adduser $USER dialout
RUN addgroup $USER dialout
RUN adduser $USER video
RUN usermod -a -G video $USER
RUN usermod -a -G render $USER

```

- Troisième étape :

Maintenant on va dans le **build_and_run.sh** et on ajoute cette ligne dans le `docker_run` :

```
--group-add #your GID \
```

Elle permet au docker de reconnaître le groupe render sur votre GID

```

docker run \
  --net=host \
  --ipc=host \
  --pid=host \
  --restart=always \
  --init \
  --ulimit core=-1 \
  --privileged \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -e DISPLAY=unix$DISPLAY \
  -v /dev:/dev \
  --group-add 992 \
  --env-file ./config/env_var.config \
  -it \
  --name $CONTAINER_NAME \

```

- Quatrième étape :

Maintenant toujours dans le **build_and_run.sh** mais dans le `docker_build`, on ajoute :

```
--build-arg RENDER_GID=$(getent group render | cut -d: -f3) \
```

Cette ligne permet de lire le fichier `/etc/group` de l'hôte et extrait le troisième champ (le GID) de la ligne du groupe render. Donc on utilise le GID de l'hôte (votre GID) automatiquement.

```

docker build -f Dockerfile -t $IMAGE_NAME . \
  --build-arg RENDER_GID=$(getent group render | cut -d: -f3) \
  --build-arg USER=$USER \
  --build-arg GROUP=$GROUP \
  --build-arg UID=$USER_UID \
  --build-arg GID=$USER_GID \
  --build-arg ABSOLUTE_WORKSPACE=$ABSOLUTE_WORKSPACE

```

- Maintenant l'accès GPU est fonctionnel

- On va maintenant installer les drivers intel pour le GPU et les bibliothèques nécessaire pour la détection d'objets. Puisqu'on modifie le build_and_run.sh, on doit le relancer à chacune de celles-ci. Or si on installe des bibliothèques directement dans le docker lorsqu'on relance le .sh les bibliothèques disparaissent complètement puisqu'elles sont stockées dans le cache du docker. Donc on va créer des dossiers externes au docker dans lesquels les bibliothèques seront installées et on appellera ces bibliothèques du docker
- Les drivers pour le GPU : L'hôte est Ubuntu 24.04 (noble). Le repo Intel direct pour noble contient libigc2 qui entre en conflit avec libigc1 déjà installé par Ubuntu. Le PPA kobuk-team est maintenu par Canonical spécifiquement pour résoudre ce problème sur noble

Explication PPA : Le PPA kobuk-team/intel-graphics est un dépôt de logiciels tiers. Le groupe kobuk-team maintient ce dépôt pour fournir des versions très récentes des pilotes graphiques Intel et des bibliothèques associées. En général, les dépôts officiels d'Ubuntu sont conservateurs : ils privilégient la stabilité et ne mettent pas à jour les pilotes vers les dernières versions disponibles. Ce PPA permet de contourner cette limitation pour Améliorer les performances dans les jeux ou les applications 3D, Bénéficier des dernières optimisations pour le décodage matériel (VA-API) ou Supporter de nouveaux matériels Intel (processeurs Core de 12e, 13e, 14e génération ou cartes graphiques Arc) qui pourraient mal fonctionner avec les anciens pilotes (comme c'est le cas ici pour nous)

- On peut donc rentrer ces commandes dans **le terminal de l'hôte** :

Ajoute le PPA Canonical pour Intel GPU

sudo add-apt-repository -y ppa:kobuk-team/intel-graphics

sudo apt update

Installe les libs

libigdgmm12 : Intel Graphics Memory Management Library

libigc2 : Intel Graphics Compiler (compile les shaders/kernels GPU)

libigdfcl2 : Intel Graphics FCL (Frontend Compiler Library)

intel-opencl-icd : driver OpenCL Intel (interface standard pour GPU computing)

libze1 : bibliothèque Level Zero (interface bas niveau Intel pour GPU/NPU)

libze-intel-gpu1 : implémentation Level Zero pour GPU Intel

```
# clinfo : outil de diagnostic pour vérifier ce qu'OpenCL voit
sudo apt install -y \
libigdgmm12 \
libigc2 \
libigdfcl2 \
intel-opencl-icd \
libze1 \
libze-intel-gpu1 \
clinfo
```

```
# Ajoute ton user aux groupes
# render : accès à /dev/dri/renderD128 (GPU compute)
# video : accès à /dev/dri/card0 (GPU display)
sudo usermod -aG render $USER
sudo usermod -aG video $USER
```

```
# Applique les groupes dans la session courante sans reboot
newgrp render
```

- En faisant la commande :

```
clinfo | grep "Device Name"
```

on voit les différent que le GPU est détecté (si plusieurs fois = normal car une fois par sous device)

```
user@grkit-009:~/roskit_ws$ clinfo | grep "Device Name"
Device Name                                Intel(R) Graphics
Device Name                                Intel(R) Graphics
Device Name                                Intel(R) Graphics
Device Name                                Intel(R) Graphics
```

- Maintenant il faut installer les librairies pip, apt et python. Il faut rentrer ces commandes dans **le terminal de l'hôte** :
 - # pip-cache : stocke les .whl téléchargés, pip ne re-télécharge pas s'ils sont là

```
mkdir -p $HOME/ros2-libs/pip-cache
```
 - # apt-cache : stocke les .deb téléchargés

```
mkdir -p $HOME/ros2-libs/apt-cache
```
 - # python-packages : les packages pip installés eux-mêmes

```
mkdir -p $HOME/ros2-libs/python-packages
```
 - Maintenant je dois modifier le **Dockerfile** pour y ajouter les librairies :
Voici les problèmes rencontrés et les solutions apportées :

Problème 1 : Le repo Intel jammy contient deux versions incompatibles des libs Intel Compiler : `libigc1` (ancienne) et `libigc2` (nouvelle). Certains paquets comme `libze-intel-gpu1` demandent `libigc2 >= 2.11.7`, mais APT essayait d'installer `libigc1` en même temps → conflit.

Solution : épingler toutes les versions sur le build **1146** (le plus récent et cohérent) :

```
libigc2=2.11.12-1146~22.04
libigdfcl2=2.11.12-1146~22.04
intel-opencl-icd=25.18.33578.15-1146~22.04
```

Le numéro **1146** est le numéro de release interne d'Intel — tous les paquets du même build sont garantis compatibles entre eux.

Problème 2 : `python3-catkin-pkg` et `pip` n'existent pas sous ces noms dans jammy.

Solution : `pip` → `python3-pip`, `python3-catkin-pkg` → `python3-catkin-pkg-modules`.

Problème 3 : Après le bloc Intel qui fait `rm -rf /var/lib/apt/lists/*`, les étapes suivantes ne trouvaient plus les paquets Ubuntu car les listes étaient effacées.

Solution : ajouter `apt-get update` au début de chaque **RUN** qui installe des paquets après ce bloc.

Maintenant qu'on sait comment régler les problèmes on va dans le **Dockerfile** et on ajoute :

```
# Drivers Intel GPU/NPU
# curl et gpg : outils pour télécharger et vérifier la clé du repo Intel
# Le repo jammy unified contient les drivers pour Ubuntu 22.04
# Les versions sont épinglées sur le build 1146 pour éviter les conflits
libigc1/libigc2
RUN apt-get update && apt-get install -y curl gpg && \
curl -fsSL https://repositories.intel.com/gpu/intel-graphics.key | \
gpg --dearmor -o /usr/share/keyrings/intel-graphics.gpg && \
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/intel-graphics.gpg] \
https://repositories.intel.com/gpu/ubuntu jammy unified" \
> /etc/apt/sources.list.d/intel-gpu-jammy.list && \
apt-get update && apt-get install -y \
libigdgmm12 \
libigc2=2.11.12-1146~22.04 \
libigdfcl2=2.11.12-1146~22.04 \
intel-opencl-icd=25.18.33578.15-1146~22.04 \
libze-intel-gpu1 \
libze1 \
```

```
clinfo \  
&& rm -rf /var/lib/apt/lists/*
```

```
# Drivers Intel GPU/NPU  
RUN apt-get update && apt-get install -y curl gpg && \  
curl -fsSL https://repositories.intel.com/gpu/intel-graphics.key | \  
gpg --dearmor -o /usr/share/keyrings/intel-graphics.gpg && \  
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/intel-graphics.gpg] \  
https://repositories.intel.com/gpu/ubuntu jammy unified" \  
> /etc/apt/sources.list.d/intel-gpu-jammy.list && \  
apt-get update && apt-get install -y \  
libigdgmm12 \  
libigc2=2.11.12-1146~22.04 \  
libigdfcl2=2.11.12-1146~22.04 \  
intel-opencl-icd=25.18.33578.15-1146~22.04 \  
libze-intel-gpu1 \  
libze1 \  
clinfo \  
&& rm -rf /var/lib/apt/lists/*  
  
# Synchronise le GID render avec l'hôte  
ARG RENDER_GID=992  
RUN groupmod -g ${RENDER_GID} render || groupadd -g ${RENDER_GID} render  
  
##### ADD NEW DEPENDENCIES JUST ABOVE THIS LINE #####
```

- Maintenant dernière modification du **Dockerfile** pour python :
Ajout juste après « ADD NEW DEPENDENCIES ... » de :
python3-pip : gestionnaire de packages Python (remplace "pip" qui
n'existe pas) # python3-catkin-pkg-modules : remplace python3-catkin-
pkg (renommé dans jammy)
RUN apt-get update && apt-get install -y \
libpython3-dev \
python3-pip \
python3-venv \
python3-catkin-pkg-modules


```
##### ADD NEW DEPENDENCIES JUST ABOVE THIS LINE #####

RUN apt-get update && apt-get install -y \
    libpython3-dev \
    python3-pip \
    python3-venv \
    python3-catkin-pkg-modules
RUN apt-get update && apt install --no-install-recommends -y libasio-dev
RUN apt-get update && apt-get install -y ros-humble-robot-state-publisher
RUN apt-get update && apt-get install -y ros-humble-joint-state-publisher-gui
RUN /usr/sbin/update-ccache-symlinks
ENV PATH="/usr/lib/ccache:${PATH}"
RUN apt-get update && apt-get install -y python3-colcon-common-extensions
RUN apt-get update && DEBIAN_FRONTEND=noninteractive apt install -y \
    ros-humble-pcl-ros \
    build-essential \
    libeigen3-dev \
    libjsoncpp-dev \
```

- Nous avons fini de modifier de Dockerfile, maintenant on retouche au **build_and_run.sh** :

On va sourcer les dossiers des caches qu'on a créé pour que le docker se serve des bibliothèques qu'il y a dedans. Comme ça à chaque fois qu'on rebuild un docker on a pas besoin de tout retélécharger, c'est stocké dans les dossiers pour toujours.

Il faut ajouter les lignes suivantes dans le docker_run :

```
-v $HOME/ros2-libs/pip-cache:/home/$USER/.cache/pip \
-v $HOME/ros2-libs/apt-cache:/var/cache/apt/archives \
-v $HOME/ros2-libs/python-
packages:/home/$USER/.local/lib/python3.10/site-packages \
```

- enfin dernière verification : il faut mettre le bon display donc toujours dans le docker run, changez **unix** par **:1** car c'est le bon display que rviz2 sera capable de voir. **-e DISPLAY=unix\$DISPLAY **

```

docker run \
  --net=host \
  --ipc=host \
  --pid=host \
  --restart=always \
  --init \
  --ulimit core=-1 \
  --privileged \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -e DISPLAY=unix$DISPLAY \
  -v /dev:/dev \
  --group-add 992 \
  --env-file ./config/env_var.config \
  -it \
  --name $CONTAINER_NAME \
  -h $CONTAINER_NAME \
  --add-host $CONTAINER_NAME:"127.0.0.1" \
  -v $HOME/.ccache:$HOME/.ccache \
  -v $HOME/ros2-libs/pip-cache:/home/$USER/.cache/pip \
  -v $HOME/ros2-libs/apt-cache:/var/cache/apt/archives \
  -v $HOME/ros2-libs/python-packages:/home/$USER/.local/lib/python3.10/site-packages \
  -v $ABSOLUTE_WORKSPACE:$ABSOLUTE_WORKSPACE \
  -e ABSOLUTE_WORKSPACE=$ABSOLUTE_WORKSPACE \
  -e ROS_DOMAIN_ID=$ROBOT_ID \
  -w $ABSOLUTE_WORKSPACE \
  $IMAGE_NAME

```

- pour build et lancer il faut exécuter le fichier build_and_run.sh, il faut donc aller dans le dossier dans lequel il est et faire les commandes suivantes :
`cd ~/roskit_ws/src/gr_docker` #on va dans le bon dossier dans le terminal de l'hôte
`./build_and_run.sh` #on exécute le fichier
- Normalement à cette étape le docker a été créé et on est à l'intérieur. Mais si vous quittez le docker en écrivant exit vous pouvez rentrer à nouveau avec la commande :
`docker exec -it gr-p247-humble-container bash`
- Puisque nous venons de créer le docker il nous faut installer les bibliothèques car c'est la première fois. Une fois installées elles seront stockées dans les dossiers créés précédemment et on aura plus besoin de le faire à nouveau.

Dans le **terminal de l'hôte**

Donne les bonnes permissions à ton user

`sudo chown -R $USER:$USER $HOME/ros2-libs/`

Dans le terminal du **docker** il faut rentrer les lignes :

Dans le conteneur — corrige les permissions sur .local

(nécessaire car le volume était monté avec des droits root au premier lancement)

```
sudo chown -R user:user /home/user/.local/
```

et aussi les commande pour enfin installer les bibliothèques :

```
pip install opencv-python ultralytics
```

- Pour vérifier que le GPU est détecté, on crée un fichier test_intel.py dans le roskit_ws et on le lance avec : `python3 test_intel.py`

```
- import opencv as cv
-
- core = cv.Core()
- devices = core.available_devices
-
- print("--- Rapport de ton Protocole ---")
- for device in devices:
-     full_name = core.get_property(device, "FULL_DEVICE_NAME")
-     print(f"Matériel détecté : {device} ({full_name})")
-
- if 'GPU' in devices:
-     print("\nSUCCÈS : Ton GPU Intel Arc est prêt pour YOLOv11 !")
- else:
-     print("\nATTENTION : Seul le CPU est visible. Vérifie tes accès Docker.")
```

- On a fini de tout mettre en place ! On peut vérifier en lançant le programme test_intel.py dans le **docker** :

```
user@gr-p247-humble-container:~/rokit_ws$ python3 test_intel.py
--- Rapport de ton Protocole ---
Matériel détecté : CPU (Intel(R) Core(TM) Ultra 7 255H)
Matériel détecté : GPU (Intel(R) Graphics (iGPU))

SUCCÈS : Ton GPU Intel Arc est prêt pour YOLOv11 !
```

Yolov11n est prêt à être utilisé !

Partie 2 :

- Pour cette partie on va mettre en place l'environnement ros2, c'est-à-dire création des packages, des codes, dependencies, etc.
- Création du package, on va créer un package de type ament python car c'est plus facile pour l'importation de Yolov11n, modèle d'IA qu'on utilisera ici :

```
ros2 pkg create --build-type ament_python detection_pkg --dependencies rclpy
sensor_msgs cv_bridge geometry_msgs vision_msgs
```

On a donc un package nommé detection_pkg dans le docker, on y placera les codes python de détection.

- On va exporter yolov11 en format openVINO, donc pour l'optimiser pour le GPU intel. Dans le terminal du **docker** :

```
python3 -c "
```

```
from ultralytics import YOLO
```

```
model = YOLO('yolo11n.pt')
```

```
model.export(format='openvino', device='CPU')
```

```
print(modele exporté dans yolo11n_openvino_model/')
```

```
"
```

```
user@gr-p247-humble-container:~/roskit_ws$ cd $ABSOLUTE_WORKSPACE/src
user@gr-p247-humble-container:~/roskit_ws/src$ ros2 pkg create --build-type ament_python detection_pkg \
--dependencies rclpy sensor_msgs cv_bridge geometry_msgs vision_msgs
going to create a new package
package name: detection_pkg
destination directory: /home/user/roskit_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['user <user@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_python
dependencies: ['rclpy', 'sensor_msgs', 'cv_bridge', 'geometry_msgs', 'vision_msgs']
creating folder ./detection_pkg
creating ./detection_pkg/package.xml
creating source folder
creating folder ./detection_pkg/detection_pkg
creating ./detection_pkg/setup.py
creating ./detection_pkg/setup.cfg
creating folder ./detection_pkg/resource
creating ./detection_pkg/resource/detection_pkg
creating ./detection_pkg/detection_pkg/__init__.py
creating folder ./detection_pkg/test
creating ./detection_pkg/test/test_copyright.py
creating ./detection_pkg/test/test_flake8.py
creating ./detection_pkg/test/test_pep257.py

[WARNING]: Unknown license 'TODO: License declaration'. This has been set in the package.xml, but no LICENSE file has been created.
It is recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
```

```

user@gr-p247-humble-container:~/roskit_ws/src$ cd ..
user@gr-p247-humble-container:~/roskit_ws$ # Dans le conteneur, n'importe où
cd $ABSOLUTE_WORKSPACE

python3 -c "
from ultralytics import YOLO
model = YOLO('yolo11n.pt') # télécharge automatiquement
model.export(format='openvino', device='CPU')
print('Modèle exporté dans yolo11n_openvino_model/')
"

WARNING ⚠️user config directory '/home/user/.config/Ultralytics' is not writable, using '/tmp/Ultralytics'. Set YOLO_CONFIG_DIR to override.
Creating new Ultralytics Settings v0.0.6 file ✅
View Ultralytics Settings with 'yolo settings' or at '/tmp/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 1% ————— 64.0KB/5.4MB 180.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 3% ————— 144.0KB/5.4MB 361.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 7% ————— 360.0KB/5.4MB 2.1
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 7% ————— 408.0KB/5.4MB 347.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 8% ————— 456.0KB/5.4MB 376.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 10% ————— 552.0KB/5.4MB 48.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 12% ————— 632.0KB/5.4MB 52.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 13% ————— 696.0KB/5.4MB 52.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 14% ————— 776.0KB/5.4MB 60.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 16% ————— 872.0KB/5.4MB 60.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 17% ————— 952.0KB/5.4MB 66.
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 19% ————— 1.0/5.4MB 652.6K
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 21% ————— 1.1/5.4MB 737.4K
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 23% ————— 1.3/5.4MB 1.1MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 27% ————— 1.4/5.4MB 1.7MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 29% ————— 1.6/5.4MB 1.1MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 32% ————— 1.7/5.4MB 977.2K
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 35% ————— 1.9/5.4MB 1.6MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 39% ————— 2.1/5.4MB 2.1MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 43% ————— 2.3/5.4MB 2.0MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 47% ————— 2.5/5.4MB 1.4MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 50% ————— 2.7/5.4MB 1.7MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 52% ————— 2.8/5.4MB 1.2MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 56% ————— 3.0/5.4MB 1.8MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 58% ————— 3.1/5.4MB 1.1MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 64% ————— 3.4/5.4MB 3.4MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 64% ————— 3.4/5.4MB 3.4MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 70% ————— 3.7/5.4MB 2.4MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 73% ————— 3.9/5.4MB 1.8MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 76% ————— 4.1/5.4MB 1.3MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 78% ————— 4.2/5.4MB 1.2MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 82% ————— 4.4/5.4MB 1.4MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 87% ————— 4.6/5.4MB 2.4MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 88% ————— 4.7/5.4MB 726.5K
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 93% ————— 5.0/5.4MB 2.4MB/
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 100% ————— 5.3/5.4MB 3.4MB
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt': 100% ————— 5.4MB 1.3MB/s 4.0s
Ultralytics 8.4.41 🐍 Python-3.10.12 torch-2.11.0+cu113 CPU (Intel Core Ultra 7 255H)
💡 ProfTip: Export to OpenVINO format for best performance on Intel hardware. Learn more at https://docs.ultralytics.com/integrations/openvino/
YOLO11n summary (fused): 100 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs

PyTorch: starting from 'yolo11n.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) (1, 84, 8400) (5.4 MB)

OpenVINO: starting export with openvino 2026.1.0-21367-63e31528c62-releases/2026/1...
OpenVINO: export success ✅ 1.4s, saved as 'yolo11n_openvino_model/' (10.4 MB)

Export complete (1.7s)
Results saved to /home/user/roskit_ws
Predict: yolo predict task=detect model=yolo11n_openvino_model/ imgsz=640
Validate: yolo val task=detect model=yolo11n_openvino_model/ imgsz=640 data=/usr/src/ultralytics/ultralytics/cfg/datasets/coco.yaml
Visualize: https://netron.app
Modèle exporté dans yolo11n_openvino_model/
user@gr-p247-humble-container:~/roskit_ws$

```

- Maintenant il faut ajouter le code python de détection : on fait premièrement de la detection 2d avec YOLO. Créer le fichier detector_2d.py dans **roskit_ws/src/detection_pkg/detection_pkg**

```

- import rclpy
- from rclpy.node import Node
- from sensor_msgs.msg import Image
- from cv_bridge import CvBridge
- import cv2
- import os
- import numpy as np
- import openvino as ov
-

```

```

- class Detector2D(Node):
-     def __init__(self):
-         super().__init__('detector_2d')
-
-         self.declare_parameter('device', 'GPU')
-         self.declare_parameter('confidence', 0.5)
-         self.declare_parameter('model_path',
-             os.path.join(os.path.dirname(__file__), '..', 'models',
- 'yolo11n_openvino_model')
-         )
-
-         device      = self.get_parameter('device').value
-         confidence  = self.get_parameter('confidence').value
-         model_path  = self.get_parameter('model_path').value
-
-         self.confidence = confidence
-         self.bridge      = CvBridge()
-
-         # Chargement via OpenVINO directement
-         self.get_logger().info(f'Chargement du modèle OpenVINO sur
- {device}...')
-         core = ov.Core()
-         model_xml = os.path.join(model_path, 'yolo11n.xml')
-         model = core.read_model(model_xml)
-         self.compiled = core.compile_model(model, device)
-         self.output    = self.compiled.output(0)
-
-         # Taille d'entrée du modèle (640x640 pour yolo11n)
-         self.input_w = 640
-         self.input_h = 640
-
-         # Noms des classes COCO
-         self.class_names = [
-             'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus', 'train', 'truck',
-             'boat', 'traffic light', 'fire hydrant', 'stop sign', 'parking meter', 'bench',
-             'bird', 'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe',
-             'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard',
-             'sports ball', 'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard',
-             'tennis racket', 'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
-             'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza',
-             'donut', 'cake', 'chair', 'couch', 'potted plant', 'bed', 'dining table', 'toilet',

```

```

-         'tv', 'laptop', 'mouse', 'remote', 'keyboard', 'cell
phone', 'microwave', 'oven',
-         'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase', 'sciss
ors', 'teddy bear',
-         'hair drier', 'toothbrush'
-     ]
-
-     self.sub = self.create_subscription(
-         Image,
-         '/camera/camera/color/image_raw',
-         self.callback,
-         10
-     )
-     self.pub_image = self.create_publisher(Image,
- '/detection/image', 10)
-
-     self.get_logger().info(f'Detector2D prêt sur {device}')
-
- def preprocess(self, frame):
-     # Resize + normalize pour YOLOv11
-     img = cv2.resize(frame, (self.input_w, self.input_h))
-     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
-     img = img.astype(np.float32) / 255.0
-     img = img.transpose(2, 0, 1)          # HWC → CHW
-     img = np.expand_dims(img, axis=0)     # CHW → NCHW
-     return img
-
- def postprocess(self, output, orig_w, orig_h):
-     # output shape : (1, 84, 8400) pour yolo11n COCO
-     preds = output[0]  # (84, 8400)
-     preds = preds.T    # (8400, 84)
-
-     boxes  = preds[:, :4]  # cx, cy, w, h
-     scores = preds[:, 4:]  # 80 classes
-
-     class_ids = np.argmax(scores, axis=1)
-     confidences = scores[np.arange(len(scores)), class_ids]
-
-     # Filtre par confiance
-     mask = confidences > self.confidence
-     boxes      = boxes[mask]
-     confidences = confidences[mask]
-     class_ids  = class_ids[mask]
-
-     # Conversion cx,cy,w,h → x1,y1,x2,y2 en coordonnées originales
-     scale_x = orig_w / self.input_w
-     scale_y = orig_h / self.input_h
-
-     results = []

```

```

-         for box, conf, cls in zip(boxes, confidences, class_ids):
-             cx, cy, w, h = box
-             x1 = int((cx - w / 2) * scale_x)
-             y1 = int((cy - h / 2) * scale_y)
-             x2 = int((cx + w / 2) * scale_x)
-             y2 = int((cy + h / 2) * scale_y)
-             results.append((x1, y1, x2, y2, float(conf), int(cls)))
-
-         return results
-
-     def callback(self, msg):
-         frame = self.bridge.imgmsg_to_cv2(msg,
desired_encoding='bgr8')
-         orig_h, orig_w = frame.shape[:2]
-
-         inp = self.preprocess(frame)
-         output = self.compiled([inp])[self.output]
-         detections = self.postprocess(output, orig_w, orig_h)
-
-         for x1, y1, x2, y2, conf, cls in detections:
-             label = self.class_names[cls] if cls < len(self.class_names)
else str(cls)
-             cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
-             cv2.putText(
-                 frame,
-                 f'{label} {conf:.2f}',
-                 (x1, y1 - 8),
-                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2
-             )
-
-         out_msg = self.bridge.cv2_to_imgmsg(frame, encoding='bgr8')
-         out_msg.header = msg.header
-         self.pub_image.publish(out_msg)
-
-     def main(args=None):
-         rclpy.init(args=args)
-         node = Detector2D()
-         rclpy.spin(node)
-         node.destroy_node()
-         rclpy.shutdown()
-
- if __name__ == '__main__':
-     main()

```

- Ce code utilise bien opencv et le GPU intégré de intel
- Il nous faut modifier le setup.py et y ajouter ce code :
- from setuptools import setup
- import os


```
- from glob import glob
-
- package_name = 'detection_pkg'
-
- setup(
-     name=package_name,
-     version='0.0.1',
-     packages=[package_name],
-     data_files=[
-         ('share/ament_index/resource_index/packages',
-          ['resource/' + package_name]),
-         ('share/' + package_name, ['package.xml']),
-         # Inclut les modèles dans l'installation
-         (os.path.join('share', package_name, 'models'),
-          glob('models/yolo11n_openvino_model/*')),
-     ],
-     install_requires=['setuptools'],
-     zip_safe=True,
-     entry_points={
-         'console_scripts': [
-             'detector_2d = detection_pkg.detector_2d:main',
-         ],
-     },
- )
```

```

src > detection_pkg > setup.py
1  from setuptools import setup
2  import os
3  from glob import glob
4
5  package_name = 'detection_pkg'
6
7  setup(
8      name=package_name,
9      version='0.0.1',
10     packages=[package_name],
11     data_files=[
12         ('share/ament_index/resource_index/packages',
13          ['resource/' + package_name]),
14         ('share/' + package_name, ['package.xml']),
15         # Inclut les modèles dans l'installation
16         (os.path.join('share', package_name, 'models'),
17          glob('models/yolo11n_openvino_model/*')),
18     ],
19     install_requires=['setuptools'],
20     zip_safe=True,
21     entry_points={
22         'console_scripts': [
23             'detector_2d = detection_pkg.detector_2d:main',
24         ],
25     },
26 )

```

- Enfin il faut rajouter des dépendances dans le package.xml :

```

- <?xml version="1.0"?>
- <?xml-model href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>
- <package format="3">
-   <name>detection_pkg</name>
-   <version>0.0.0</version>
-   <description>TODO: Package description</description>
-   <maintainer email="user@todo.todo">user</maintainer>
-   <license>TODO: License declaration</license>
-
-   <depend>rclpy</depend>
-   <depend>sensor_msgs</depend>
-   <depend>cv_bridge</depend>
-
-   <test_depend>ament_copyright</test_depend>
-   <test_depend>ament_flake8</test_depend>

```

```

-   <test_depend>ament_pep257</test_depend>
-   <test_depend>python3-pytest</test_depend>
-
-   <export>
-       <build_type>ament_python</build_type>
-   </export>
- </package>

```

```

src > detection_pkg > package.xml
1  <?xml version="1.0"?>
2  <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3  <package format="3">
4      <name>detection_pkg</name>
5      <version>0.0.0</version>
6      <description>TODO: Package description</description>
7      <maintainer email="user@todo.todo">user</maintainer>
8      <license>TODO: License declaration</license>
9
10     <depend>rclpy</depend>
11     <depend>sensor_msgs</depend>
12     <depend>cv_bridge</depend>
13
14     <test_depend>ament_copyright</test_depend>
15     <test_depend>ament_flake8</test_depend>
16     <test_depend>ament_pep257</test_depend>
17     <test_depend>python3-pytest</test_depend>
18
19     <export>
20         <build_type>ament_python</build_type>
21     </export>
22 </package>
23

```

- Tous les codes et dépendances ont été mis en place, passons à la procédure pour lancer le code et les nœuds :

Dans un terminal de l'hôte, rentrez la commande : `sudo lsof /dev/video*`
`2>/dev/null`

Elle permet de voir quel groupe id, donc capteur/composant est connecté à l'hôte. Nous voulons que la caméra D45 du robot que nous utilisons soit connectée au docker et non à l'hôte. Donc si en tapant cette commande vous voyez que la caméra apparaitre c'est qu'elle est connectée à l'hôte. Il faut kill cette connexion avec la commande : `sudo kill -9 #GID de la connexion`
DANS CE MEME TERMINAL, rentrer dans le docker.

On se mets dans le docker, on rappelle qu'il faut toujours sourcer ros au lancement : `source /opt/ros/humble/setup.bash`

Maintenant on peut utiliser la commande `cm` qui permet de colcon build tous les packages du robots (cm est un alias, on peut retrouver sa signification exacte dans le bashrc)

Si lors de la compilation ça ne marche pas a cause de numpy il faut le downgrade avec la commande : `pip install "numpy<2"`

On peut maintenant lancer la première node de la caméra je rappelle dans le même container dans lequel on a kill la connexion à l'hôte.

```
cd $ABSOLUTE_WORKSPACE
```

```
source $ABSOLUTE_WORKSPACE/install/setup.bash
```

```
ros2 launch realsense2_camera rs_launch.py
```

il y aura normalement un message du docker qui indique que la caméra publie bien :

```
[realsense2_camera_node-1] [INFO] [1776929801.222897770] [camera.camera]: RealSense Node Is Up!
```

- On ouvre un autre terminal, on rentre dans le docker et on lance la deuxième node (pas besoin de ressource ou de rebuild vu qu'on est dans le docker et qu'on l'a déjà fait sur l'autre terminal) :

```
source $ABSOLUTE_WORKSPACE/install/setup.bash
```

```
ros2 run detection_pkg detector_2d --ros-args -p device:=GPU
```

La node qu'on a créée avec notre code s'abonne au topic de la caméra et lance la détection.

- Lancer noVNC sur internet, se connecter au docker. Si rviz2 est déjà lancé, vous pouvez cliquer en bas à gauche add -> by topic -> detection/image/image et la fenêtre avec **la détection se lance !**

Si rviz2 n'est pas lancé vous pouvez dans un terminal docker de noVNC ou de vs code taper la commande :

```
source $ABSOLUTE_WORKSPACE/install/setup.bash
```

```
ros2 run rviz2 rviz2
```

Il est possible que en lançant cette ligne il y ait des messages d'erreur :

```
user@gr-p247-humble-container:~/roskit_ws$ ros2 run rviz2 rviz2
```

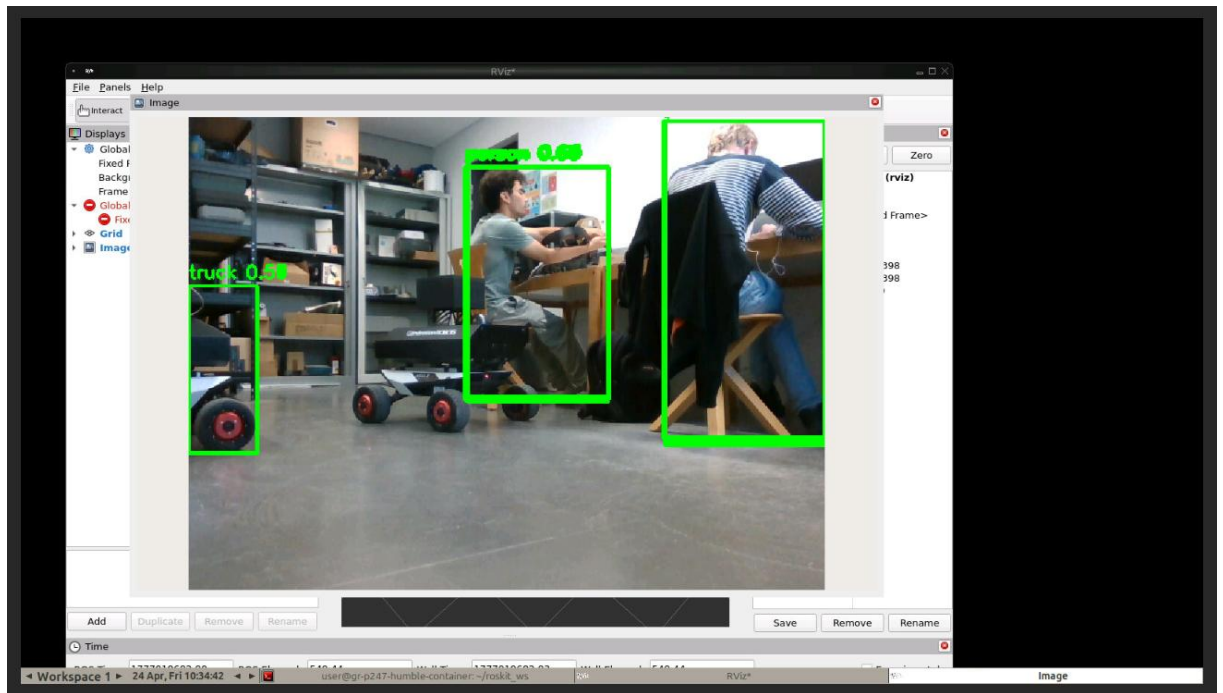
```
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-user'
```

```
Failed to create /home/user/.cache/mesa_shader_cache for shader cache  
(Permission denied)---disabling.
```

```
[INFO] [1776936262.836120801] [rviz2]: Stereo is NOT SUPPORTED
```

```
[INFO] [1776936262.836173312] [rviz2]: OpenGL version: 4.5 (GLSL 4.5)
```

Ces messages n'impactent pas le fonctionnement du programme



Problèmes et solutions :

- Si pendant le lancement du build_and_run.sh avec la commande `./build_and_run.sh` dans l'hôte, dans le bon dossier, vous avez une erreur avec xhost il faut rajouter `|| true` après le xhost du build_and_run.sh
- Si problème lors du lancement de la node de detection de la caméra, voir la procédure de lancement
- Si problème de version numpy il faut downgrade sa version `<1` car car cv_bridge et compilé avec NumPy 1.x
- Si le terminal de l'hôte n'a plus de couleur et que l'auto completion du docker ne marche plus, c'est un léger problème de compatibilité avec les drivers intel de PPA, il suffit juste d'écrire la commande à la main : `ros2 exec -it gr-p247-humble-container bash`
- Si problème de display lors du lancement de rviz2, voir procédure de lancement. Si toujours un problème la commande `export DISPLAY=:1` permet de connecter la bon display au container.

Liste commandes :

- Avec la commande : `cat /etc/group` on peut voir tous les groupes du système. Avec la commande : `stat /dev/dri/renderD128` on peut voir plus en détail ce groupe
- `clinfo | grep "Device Name"` permet de voir les devices connecté à OpenCL

- Commande pour voir tous les .msg de ros2 humble et du ws :
`find / -name "*.msg" 2>/dev/null`
et
Tes messages custom dans le workspace
`find $ABSOLUTE_WORKSPACE/src -name "*.msg"`

Les messages ROS2 installés
`find /opt/ros/humble -name "*.msg"`
- `echo $DISPLAY` #donne la liste des display, tu verras normalement rien, unix ou 1
-

A VERIFIER

Qu'y a-t-il dans le setup.py, package.xml quand on vient de les créer ou savoir si juste je rajoute ou je les fait copier-coller

Vérifier si la camera se connecte à l'hôte uniquement après avoir cm ou alors avant aussi car ça changera l'ordre des deux premières étapes de la procédure de lancement du code de détection.